FGV EMAp

João Pedro Jerônimo

Engenharia de Software

Revisão para A2

Rio de Janeiro

2026

Nota: Os conteúdos sobre Design Patterns foram retirados e adaptados/traduzidos da página Refactoring Guru em conjunto com códigos elaborados por conta própria e/ou retirados das aulas do professor. O conteúdo sobre SOLID foi retirado e adaptado das aulas e de sites diversos na internet.

Conteúdo

1 SOLID - L e I 3
1.1 L - Princípio da Substituição de Liskov 4
1.2 I - Princípio da Segregação de Interfaces 5
2 Chain of Responsibilities Pattern 7
2.1 O Problema 8
2.2 A Solução 9
2.3 A Variante 11
3 Strategy Pattern 13
3.1 O Problema 14
3.2 A Solução 14
4 Command Pattern 16
4.1 O Problema 17
4.2 A Solução 18
5 State Pattern 21
5.1 O Problema 22
5.2 A Solução 23

SOLID - L e I

1.1 L - Princípio da Substituição de Liskov

O Princípio da Substituição de Liskov (Liskov Substitution Principle - LSP) é um dos cinco princípios do SOLID. O LSP afirma que objetos de uma classe derivada devem poder substituir objetos da classe base **sem alterar a corretude do programa**. Em outras palavras, se uma classe S é uma subclasse de T, então objetos de tipo T podem ser substituídos por objetos de tipo S sem afetar o comportamento esperado do programa.

Imagine que uma empresa criou, internamente, um programa para atender a **todos** os seus usuários. Obviamente, ela vai utilizar da classe base `Usuario` para essa representação. No entanto, em uma empresa madura, com toda certeza vai existir algum tipo de **especialização** de usuários para atender às necessidades da empresa. Para continuar o exemplo, vamos supor a seguinte estrutura:

users.py — python

```python
class Usuario(ABC):
  @abstractmethod
  def gerar_relatorio(self):
    ...

class UsuarioComercial(Usuario):
  def gerar_relatorio(self):
    print("Gerando relatório comercial...")

class UsuarioFinanceiro(Usuario):
  def gerar_relatorio(self):
    raise NotImplementedError("Usuário financeiro não pode gerar relatórios.")
```

Essa estrutura de usuários por si só já é ruim, mas não vamos focar nisso agora. Vamos então montar uma classe para o programa dos nossos usuários!

programa.py — python

```python
class Programa:
  def __init__(self, usuario: Usuario):
    self.usuario = usuario

  def gerar_relatorio(self):
    if isinstance(self.usuario, UsuarioComercial):
      self.usuario.gerar_relatorio()
```

Como eu disse anteriormente, o programa foi feito para atender a todos os usuários, mas ele só leva em conta o usuário comercial. Em primeira vista, podemos até pensar: “Ah, mas o usuário financeiro não precisa gerar relatórios, então tudo bem”. No entanto, se criarmos uma nova classe de usuário, como `UsuarioMarketing` que precisa gerar relatórios, o programa vai quebrar, pois ele não vai saber lidar com esse novo tipo de usuário. Justamente aí que entra a violação do LSP, pois o usuário de marketing pode gerar relatórios, ele é uma subclasse de `Usuario`, então ele poderia e deveria entrar no programa sem o quebrar, mas o programa não foi feito para lidar com ele, então ele quebra. O LSP é violado.

Para consertar a violação do LSP **parcialmente**, vamos fazer uma simples alteração no programa para que ele aceite qualquer tipo de usuário:

programa.py

```python
class Programa:
  def __init__(self, usuario: Usuario):
    self.usuario = usuario

  def gerar_relatorio(self):
    self.usuario.gerar_relatorio()
```

Agora nosso programa está idealmente correto, mas o LSP **ainda é violado**. Mas por quê? Perceba que no nosso arquivo `users.py`, a classe `UsuarioFinanceiro` **não implementa o método** `gerar_relatorio()`, então se passarmos um objeto dessa classe para o programa, ele vai quebrar. O LSP é violado porque a subclasse `UsuarioFinanceiro` não pode substituir a superclasse `Usuario` sem quebrar o programa. No entanto, o LSP está sendo violado em conjunto com outro pricípio do SOLID. Antes de implementarmos a devida correção, vamos falar sobre o outro princípio que está sendo violado.

1.2 I - Princípio da Segregação de Interfaces

O Princípio da Segregação de Interfaces (Interface Segregation Principle - ISP) é outro princípio do SOLID que afirma que **nenhum cliente deve ser forçado a depender de métodos que não utiliza**. Em outras palavras, é melhor ter várias interfaces específicas do que uma interface única e genérica. O ISP incentiva a criação de interfaces menores e mais focadas, permitindo que os clientes implementem apenas os métodos que realmente precisam, evitando a implementação de métodos desnecessários.

Vamos rever o nosso arquivo `users.py` para entender como o ISP está sendo violado:

users.py

```python
class Usuario(ABC):
  @abstractmethod
  def gerar_relatorio(self):
    ...

class UsuarioComercial(Usuario):
  def gerar_relatorio(self):
    print("Gerando relatório comercial...")

class UsuarioFinanceiro(Usuario):
  def gerar_relatorio(self):
    raise NotImplementedError("Usuário financeiro não pode gerar relatórios.")
```

Nós haviamos dito anteriormente que **todos** os usuários deveriam ter acesso ao nosso programa! Para isso, precisamos que todos eles possam gerar relatório, entretanto, nosso usuário financeiro não pode gerar relatórios. **Vamos supor** que em nosso cenário, isso foi erro de modelagem do programa, e usuários financeiros **realmente não podem gerar relatórios**. O ISP está sendo violado pois, mesmo que o usuário financeiro não precise gerar relatórios, ele é forçado a implementar o método `gerar_relatorio()`, mesmo que não faça sentido para ele. Para corrigir

isso, vamos criar uma nova interface chamada `UsuarioComRelatorio`, e o método de geração de relatórios vai ser movido para essa nova interface. Assim, apenas os usuários que realmente precisam gerar relatórios vão implementar esse método.

users.py — python

```python
class Usuario(ABC):
  pass

class UsuarioComRelatorio(Usuario):
  @abstractmethod
  def gerar_relatorio(self):
    ...

class UsuarioComercial(UsuarioComRelatorio):
  def gerar_relatorio(self):
    print("Gerando relatório comercial...")

class UsuarioFinanceiro(Usuario):
  pass
```

Então mudamos a abordagem do nosso programa, em vez de ele ficar disponível para **qualquer usuário**, ele vai ficar disponível apenas para **usuários que podem gerar relatórios**.

programa.py — python

```python
class Programa:
  def __init__(self, usuario: UsuarioComRelatorio):
    self.usuario = usuario

  def gerar_relatorio(self):
    self.usuario.gerar_relatorio()
```

Dessa forma, o LSP e o ISP foram corrigidos, e nosso programa agora está idealmente correto. O LSP foi corrigido pois agora qualquer subclasse de `UsuarioComRelatorio` pode substituir a super-classe sem quebrar o programa, e o ISP foi corrigido pois agora apenas os usuários que realmente precisam gerar relatórios implementam o método `gerar_relatorio()`, evitando a implementação de métodos desnecessários.

Chain of Responsibilities Pattern

2.1 O Problema

Imagine que estamos fazendo um programa de gestão de pedidos de uma empresa. Nós temos o pedido do nosso cliente, no entanto, algumas decisões de negócio nos foram informadas. Precisamos que, antes de enviar o pedido ao sistema, alguns pré-processamentos e verificações devem ser feitos. O nosso chefe disse que devemos implementar uma verificação de que as **informações do cliente estão no pedido** e que o usuário tem permissões para realizar aquele pedido.

Após algumas indagações, faz sentido implementar essas verificações sequencialmente e, se alguma falhar, nem continua!

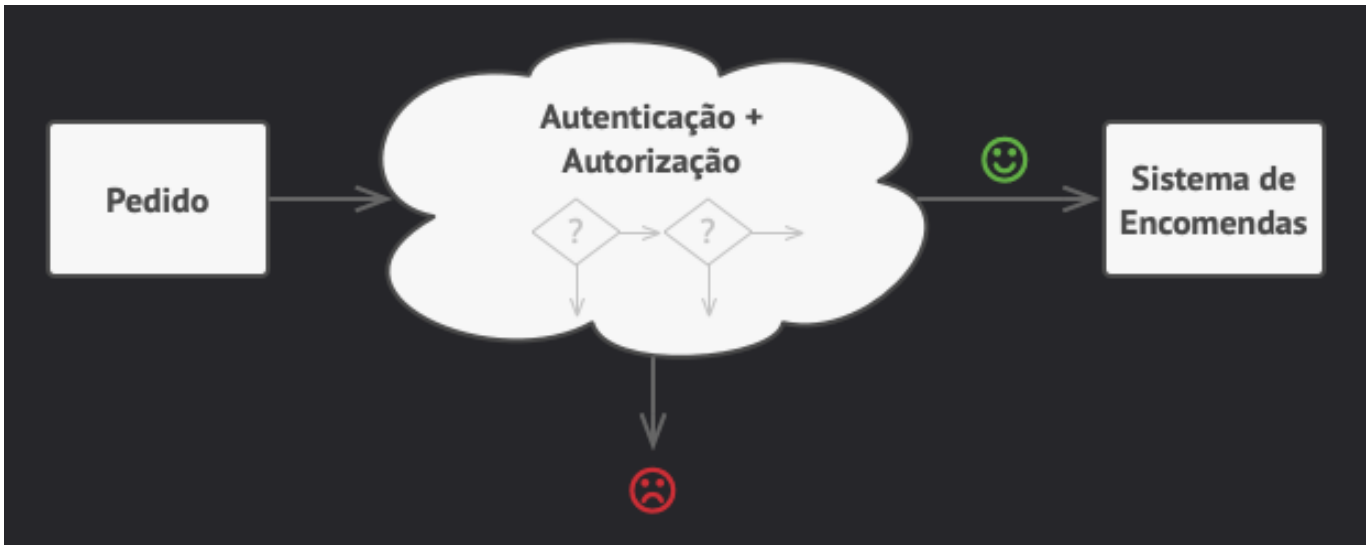


Figura 1: Diagrama do sistema com as verificações de negócio implementadas sequencialmente

Vamos criar essa estrutura então!

```python
@dataclass
class Pedido:
  cliente: Cliente
  itens: list[Item]
  valor_total: float

class SistemaDePedidos():
  def verificar_informacoes_cliente(self, cliente: Cliente) -> bool:
    # Lógica para verificar informações do cliente
    ...

  def verificar_permissoes_usuario(self, usuario: Usuario) -> bool:
    # Lógica para verificar permissões do usuário
    ...

  def processar_pedido(self, pedido: Pedido):
    if not self.verificar_informacoes_cliente(pedido.cliente):
      raise ValueError("Informações do cliente inválidas.")
    if not self.verificar_permissoes_usuario(pedido.cliente.usuario):
      raise ValueError("Usuário não tem permissões para realizar o pedido.")
    ...
```

Caraca, que código lindo... Agora vamos imaginar a seguinte situação: Imagine que, conversando com seu colega de trabalho, ele fala que faz sentido adicionar uma verificação para que o pedido não ultrapasse o limite de crédito do cliente. Porém, também faz sentido adicionar uma checagem de segurança contra brute force attacks... O que você acha que deveríamos fazer dentro dessa classe que criamos para adicionar esses pontos que nosso colega de trabalho citou? Exatamente, **modificar** a classe `SistemaDePedidos`.

Pelo que já vimos desde os conceitos iniciais do SOLID, essa classe de sistema de pedidos tá uma **porcaria**. Primeiro que ela viola o *S* ao colocar a responsabilidade de fazer as verificações no `SistemaDePedidos` e ela viola o *O* ao fazer que, se eu quiser adicionar uma nova verificação, eu vou ter que modificar a classe `SistemaDePedidos`. Então o que cacete vamos fazer pra corrigir isso? É aí que entra o Chain of Responsibilities Pattern.

2.2 A Solução

O design define que **cada checagem** deve ser feita de forma **independente** por uma **classe separada** chamada **handler**, de forma que existe uma função própria que, ao ser chamada, faz a checagem e, caso ela passe, chama o próximo handler da cadeia. Caso a checagem falhe, o handler lança uma exceção e a cadeia é interrompida. Dessa forma, cada handler é responsável por uma única checagem e a responsabilidade de chamar o próximo handler é delegada para o próprio handler, além de que, se um falha, a cadeia é interrompida e o pedido não é processado.

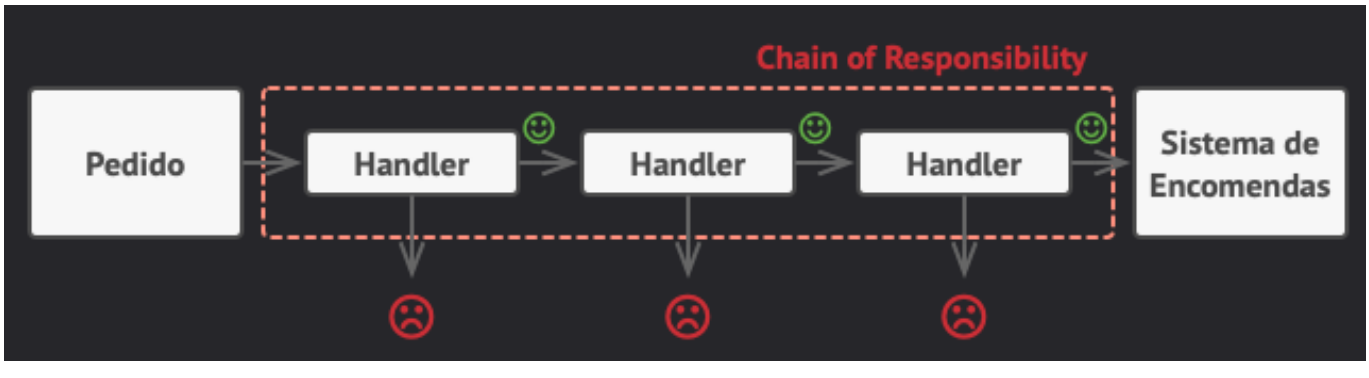


Figura 2: Diagrama do sistema com as verificações de negócio implementadas em handlers independentes

Vamos então corrigir aquele código feio que fizemos anteriormente e implementar o Chain of Responsibilities Pattern.

pedido.py — python

```python
@dataclass
class Pedido:
  cliente: Cliente
  itens: list[Item]
  valor_total: float
```

handlers.py — python

```python
class Handler(ABC):
  def __init__(self):
    self._proximo_handler = None

  def set_proximo(self, handler: "Handler"):
```

```python
    self._proximo_handler = handler
    return handler  # Permite fazer encadeamento de chamadas

  @abstractmethod
  def verificar(self, pedido: Pedido):
    pass


class InformacaoClienteHandler(Handler):
  def verificar(self, pedido: Pedido):
    if not condition:  # Substitua "condition" pela lógica de checagem real
      raise ValueError("Informações do cliente inválidas.")
    if self._proximo_handler:
      self._proximo_handler.verificar(pedido)


class PermissaoUsuarioHandler(Handler):
  def verificar(self, pedido: Pedido):
    if not condition:  # Substitua "condition" pela lógica de checagem real
      raise ValueError("Permissão de usuário inválida.")
    if self._proximo_handler:
      self._proximo_handler.verificar(pedido)


class LimiteCreditoHandler(Handler):
  def verificar(self, pedido: Pedido):
    if not condition:  # Substitua "condition" pela lógica de checagem real
      raise ValueError("Limite de crédito insuficiente.")
    if self._proximo_handler:
      self._proximo_handler.verificar(pedido)


class BruteForceHandler(Handler):
  def verificar(self, pedido: Pedido):
    if not condition:  # Substitua "condition" pela lógica de checagem real
      raise ValueError("Possível ataque de força bruta detectado.")
    if self._proximo_handler:
      self._proximo_handler.verificar(pedido)
```

Com essa nova estrutura, mantemos todos os princípios do SOLID, pois cada handler é responsável por uma única checagem e, para criar uma nova verificação, basta criar um novo handler e adicioná-lo à cadeia, sem precisar modificar nenhuma das classes existentes. Vamos ver como ficaria a lógica de negócio do nosso sistema de pedidos com essa nova estrutura.

main.py — python

```python
pedido = Pedido(cliente=cliente, itens=itens, valor_total=valor_total)
```

```
2
3  # Configurar a cadeia de responsabilidade
4  informacao_cliente_handler = InformacaoClienteHandler()
5  permissao_usuario_handler = PermissaoUsuarioHandler()
6  limite_credito_handler = LimiteCreditoHandler()
7  brute_force_handler = BruteForceHandler()
8
9  informacao_cliente_handler.set_proximo(permissao_usuario_handler)\
10   .set_proximo(limite_credito_handler)\
11   .set_proximo(brute_force_handler)
12
13 informacao_cliente_handler.verificar(pedido)  # Inicia a cadeia de verificações
14
15 sistema_de_pedidos.processar_pedido(pedido)  # Processa o pedido se todas as
   verificações passarem
```

Dá até uma sensação de alívio ver que agora o código está mais organizado e fácil de manter.

2.3 A Variante

É interessante notar que o Chain of Responsibilities Pattern pode ser implementado de diferentes formas. Uma variante interessante é uma que troca a forma como a cadeia se comporta. Anteriormente, vimos que **cada handler** faz sua checagem e, caso ela passe, chama o próximo handler da cadeia. No entanto, existe uma outra abordagem em que o handler olha e diz: *"Eu posso processar? Se sim, vou processar e a cadeia **vai terminar**, se não, eu vou ver se o próximo handler consegue"*.

Pra manter a simplicidade do documento, vamos imaginar o seguinte cenário e tentar implementá-lo com essa variante do Chain of Responsibilities Pattern. Imagine que temos um sistema de atendimento de reclamações de usuários, nós temos uma classe `Reclamacao` que representa as informações que um usuário trouxe sobre seu problema. Na nossa empresa, possuímos o seguinte processo de atendimento: *Se a reclamação for simples, o atendente automático vai resolver, se for um pouco mais complexa, o atendente humano vai resolver, e se for muito complexa, o gerente vai resolver*. Então vamos implementar essa lógica com a variante do Chain of Responsibilities Pattern.

`reclamacao.py` `python`

```
1  @dataclass
2  class Problema:
3    descricao: str
4    complexidade: int  # 1 = simples, 2 = média, 3 = complexa
5
6
7  class Handler(ABC):
8    def __init__(self):
9      self._parent_handler = None
10
11   def set_parent(self, handler: "Handler"):
12     self._parent_handler = handler
13     return handler  # Permite fazer encadeamento de chamadas
```

```
14
15    @abstractmethod
16    def pode_processar(self, problema: Problema) -> bool:
17      pass
18
19    @abstractmethod
20    def processar(self, problema: Problema):
21      pass
22
23
24  class AtendenteAutomaticoHandler(Handler):
25    def pode_processar(self, problema: Problema) -> bool:
26      return problema.complexidade == 1
27
28    def processar(self, problema: Problema):
29      if not self.pode_processar(problema) and self._parent_handler:
30        self._parent_handler.processar(problema)
31        return
32      print("Atendente automático resolveu o problema.")
33
34
35  class AtendenteHumanoHandler(Handler):
36    def pode_processar(self, problema: Problema) -> bool:
37      return problema.complexidade == 2
38
39    def processar(self, problema: Problema):
40      if not self.pode_processar(problema) and self._parent_handler:
41        self._parent_handler.processar(problema)
42        return
43      print("Atendente humano resolveu o problema.")
44
45
46  class GerenteHandler(Handler):
47    def pode_processar(self, problema: Problema) -> bool:
48      return problema.complexidade == 3
49
50    def processar(self, problema: Problema):
51      if not self.pode_processar(problema) and self._parent_handler:
52        self._parent_handler.processar(problema)
53        return
54      print("Gerente resolveu o problema.")
```

Nessa lógica, cada um dos handlers verifica se consegue processar o problema. Caso consiga, ele processa e a cadeia termina. Caso não consiga, ele passa para o próximo handler da cadeia.

Strategy Pattern

3.1 O Problema

Vamos voltar ao problema da criação de um sistema de pedidos na minha empresa. No entanto, vamos supor que agora estamos trabalhando na área de **cálculo do frete do pedido**. A empresa possui diferentes tipos de frete. São três: Frete Normal, Frete Expresso e Frete Grátis.

```python
@dataclass
class Pedido:
  cliente: Cliente
  peso_kg: float
  valor: float


class CalculadoraFrete:
  def calcular_frete(self, pedido: Pedido, frete_tipo: str) -> float:
    if frete_tipo == "normal":
      return 10 + pedido.peso_kg * 2  # Exemplo de cálculo para frete normal
    elif frete_tipo == "expresso":
      return 25 + pedido.peso_kg * 4  # Exemplo de cálculo para frete expresso
    elif frete_tipo == "gratis":
      return 0.0  # Frete grátis
    else:
      raise ValueError("Tipo de frete inválido.")
```

Dado todas as conversas que tivemos durante o documento, você já consegue ver como esse código que fizemos chega a dar calafrios. Ele viola o *S* pois a responsabilidade de calcular o frete está na classe `CalculadoraFrete`, e ele viola o *O* pois, se eu quiser adicionar um novo tipo de frete, vou ter que modificar a classe `CalculadoraFrete`. Então vamos corrigir isso com o Strategy Pattern.

3.2 A Solução

No Strategy Pattern, nós criamos uma classe separada para **cada tipo de frete**, cada uma dessas classes é chamada de **estratégia**, pois elas falam a forma específica que o problema de **calcular o frete** é dividido e atacado. Dessa forma, cada estratégia é responsável por uma única forma de calcular o frete, e a responsabilidade de escolher qual estratégia utilizar é delegada para a classe `CalculadoraFrete`, que vai receber a estratégia como parâmetro

```python
class EstrategiaFrete(ABC):
  @abstractmethod
  def calcular(self, pedido: Pedido) -> float:
      ...


class FreteNormal(EstrategiaFrete):
  def calcular(self, pedido: Pedido) -> float:
      return 10 + pedido.peso_kg * 2


class FreteExpresso(EstrategiaFrete):
  def calcular(self, pedido: Pedido) -> float:
```

```python
        return 25 + pedido.peso_kg * 4


class FreteGratis(EstrategiaFrete):
  def calcular(self, pedido: Pedido) -> float:
        return 0.0


class CalculadoraFrete:
  """Contexto: depende só da abstração, recebida por injeção."""

  def __init__(self, estrategia: EstrategiaFrete) -> None:
        self._estrategia = estrategia

  def set_estrategia(self, estrategia: EstrategiaFrete) -> None:
        self._estrategia = estrategia

  def calcular(self, pedido: Pedido) -> float:
        return self._estrategia.calcular(pedido)
```

E aqui está um exemplo de como ficaria a utilização do Strategy Pattern para calcular o frete de um pedido:

```python
pedido = Pedido("Carlinho", peso_kg=3, valor=200)

calc = CalculadoraFrete(FreteNormal())
print(f"Normal:   R$ {calc.calcular(pedido):.2f}")   # 16.00

calc.set_estrategia(FreteExpresso())
print(f"Expresso: R$ {calc.calcular(pedido):.2f}")   # 37.00

calc.set_estrategia(FreteGratis())
print(f"Grátis:   R$ {calc.calcular(pedido):.2f}")   # 0.00
```

Se esse exemplo ainda deixou uma pulga na sua orelha. Imagine que você quer ir de um ponto A até um ponto B, por exemplo, quer sair da Praia de Copacabana e ir até o Cristo Redentor. Existem várias formas de fazer isso, você pode ir de carro, de ônibus, de metrô, de bicicleta, a pé... Cada uma dessas formas é uma estratégia diferente para **realizar um mesmo objetivo**, que é sair de onde você está e ir até seu lugar de interesse. O Strategy Pattern é exatamente isso, ele define uma família de algoritmos, encapsula cada um deles e os torna intercambiáveis. Dessa forma, o algoritmo pode variar independentemente dos clientes que o utilizam.

Command Pattern

4.1 O Problema

Imagine que estamos fazendo um editor de documentos, e queremos implementar algumas funcionalidades como escrever, apagar, etc. No entanto, queremos que o usuário tenha a possibilidade de desfazer e refazer as ações que ele realizou. Para isso, vamos criar uma classe `Documento` que representa o documento que o usuário está editando, e uma classe `Editor` que representa o editor de documentos. A classe de `Documento` não vai ter uma implementação concreta para que possamos focar na implementação do Command Pattern.

```python
class Editor:
    def __init__(self, doc: Documento) -> None:
        self.doc = doc
        # Cada alteração vira uma tupla (acao, dado) com o necessário para desfazê-la.
        self.historico: list[tuple[str, str]] = []
        self.historico_refazer: list[tuple[str, str]] = []

    def escrever(self, trecho: str) -> None:
        self.doc.escrever(trecho)
        self.historico.append(("escrever", trecho))

    def apagar(self, n: int) -> None:
        apagado = self.doc.apagar(n)
        self.historico.append(("apagar", apagado))

    def salvar(self) -> None:
        self.doc.salvar()

    def desfazer(self) -> None:
        if not self.historico:
            return
        acao, dado = self.historico.pop()
        if acao == "escrever":
            self.doc.apagar(len(dado))
        elif acao == "apagar":
            self.doc.escrever(dado)
        self.historico_refazer.append((acao, dado))

    def refazer(self) -> None:
        if not self.historico_refazer:
            return
        acao, dado = self.historico_refazer.pop()
        if acao == "escrever":
            self.doc.escrever(dado)
        elif acao == "apagar":
            self.doc.apagar(len(dado))
```

Novamente, temos uma classe **repleta** de problemas. Ela viola o S pois a classe do editor possui múltiplas responsabilidades, como escrever no documento, apagar, refazer, desfazer, gerenciar o histórico e assim por diante. Ela também viola o *O* pois, se eu quiser adicionar uma nova ação, como copiar e colar, vou ter que modificar a classe `Editor`. Para corrigir essas problemáticas, utilizamos do Command Pattern.

4.2 A Solução

O Command Pattern encapsula cada comando como um objeto próprio, permitindo que você parametrize os comandos do editor de forma que cada comando fique responsável por uma única ação e por conhecer **como ele mesmo deve ser desfeito**. Dessa forma, o editor não precisa conhecer os detalhes de cada comando, ele apenas executa o comando e adiciona ele ao histórico.

Antes de entrarmos na implementação em si, vamos adicionar mais um tipo de comando, os comandos **que não podem ser desfeitos**. Para simplificar, vamos implementar apenas um, que é o **comando de salvar o documento**. O comando de salvar não pode ser desfeito, pois uma vez que o documento é salvo, não há como voltar atrás. Então vamos implementar a estrutura do Command Pattern para o nosso editor de documentos.

Além disso, também vamos implementar uma classe própria para o gerenciamento do histórico de comandos, que vai ser responsável por armazenar os comandos executados e permitir que o usuário desfaça e refaça as ações realizadas.

commands.py — python

```python
class Comando(ABC):
  @abstractmethod
  def executar(self) -> None:
    ...

class ComandoReversível(Comando):
  @abstractmethod
  def desfazer(self) -> None:
    ...

class EscreverComando(ComandoReversível):
  def __init__(self, doc: Documento, trecho: str) -> None:
    self.doc = doc
    self.trecho = trecho

  def executar(self) -> None:
    self.doc.escrever(self.trecho)

  def desfazer(self) -> None:
    self.doc.apagar(len(self.trecho))

class ApagarComando(ComandoReversível):
  def __init__(self, doc: Documento, quantidade: int) -> None:
    self.doc = doc
    self.quantidade = quantidade
```

```
26      self.apagado = ""
27
28    def executar(self) -> None:
29      self.apagado = self.doc.apagar(self.quantidade)
30
31    def desfazer(self) -> None:
32      self.doc.escrever(self.apagado)
33
34  class SalvarComando(Comando):
35    def __init__(self, doc: Documento) -> None:
36      self.doc = doc
37
38    def executar(self) -> None:
39      self.doc.salvar()
```

Agora que definimos toda a estrutura de como nossos comandos funcionam, vamos implementar a classe própria para gerenciar o histórico de comandos.

history.py — python

```
1   class Historico:
2     def __init__(self) -> None:
3       self.feitos: list[ComandoDesfazivel] = []
4       self.desfeitos: list[ComandoDesfazivel] = []
5
6     def registrar(self, cmd: ComandoDesfazivel) -> None:
7       self.feitos.append(cmd)
8       self.desfeitos.clear()
9
10    def desfazer(self) -> None:
11      if self.feitos:
12        cmd = self.feitos.pop()
13        cmd.desfazer()
14        self.desfeitos.append(cmd)
15
16    def refazer(self) -> None:
17      if self.desfeitos:
18        cmd = self.desfeitos.pop()
19        cmd.executar()
20        self.feitos.append(cmd)
```

editor.py — python

```
1   class Editor:
2     def __init__(self) -> None:
3       self.historico = Historico()
4
5     def executar(self, cmd: Comando) -> None:
```

```
6       cmd.executar()
7       if isinstance(cmd, ComandoDesfazivel):
8           self.historico.registrar(cmd)
9
10    def desfazer(self) -> None:
11      self.historico.desfazer()
12
13    def refazer(self) -> None:
14      self.historico.refazer()
```

Esse código ainda possui alguns problemas principalmente na **forma que os comandos lidam com o documento**, no entanto, eles não vão ser abordados nesse documento, pois o foco é apenas a implementação do Command Pattern. O que importa é que agora cada comando é responsável por uma única ação e por conhecer como ele mesmo deve ser desfeito, e o editor não precisa conhecer os detalhes de cada comando, ele apenas executa o comando e adiciona ele ao histórico. Para adicionar um novo comando, basta criar uma nova classe que herde de `Comando` ou `ComandoReversível`, sem precisar modificar a classe `Editor`. Dessa forma, o código está mais organizado e fácil de manter.

State Pattern

5.1 O Problema

Em muitas aplicações, pode surgir que, dentro de uma mesma classe, múltiplos comportamentos podem ser implementados, e isso é uma decisão de negócio válida. Por um exemplo, um documento pode ter comportamentos diferentes de acordo em **qual etapa de publicação** ele está na minha rede social. O State Pattern entra como modelar esse problema de estados distintos de forma a manter a estrutura SOLID que tanto desejamos. Podemos imaginar os problemas que ele aborda como **máquinas de estado finito**, onde representamos os estados possíveis como nós e as setas que ligam esses nós indicam as transições possíveis entre os estados.

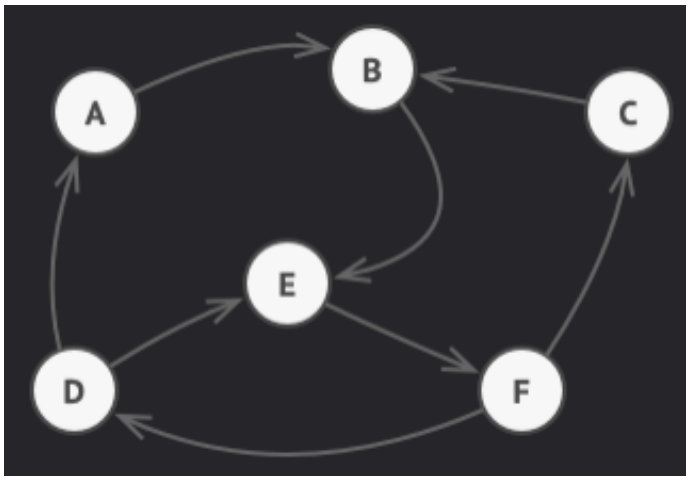


Figura 3: Diagrama de máquina de estados finita

A ideia principal é que, em qualquer dado momento, há um número finito de estados que um programa possa estar. Dentro de qualquer estado único, o programa se comporta de forma diferente, e o programa pode ser trocado de um estado para outro instantaneamente. Contudo, dependendo do estado atual, o programa pode ou não trocar para outros estados. Essas regras de troca, chamadas transições, também são finitas e pré determinadas.

Poderíamos colocar toda essa gestão em uma única classe, mas provavelmente teríamos if statements **gigantescos** (eu já fiz isso no meu último emprego) e garanto como isso gera uma **dor de cabeça a longo prazo**

Vamos trabalhar em cima do cenário do documento e que ele pode estar em três estados distintos: `Rascunho`, `Revisao` e `Publicado`. Cada estado tem comportamentos diferentes, e o documento pode transitar entre esses estados de acordo com regras de negócio específicas. As regras são as seguintes:

- Um documento em `Rascunho` pode ser enviado para `Revisao`.
- Um documento em `Revisao` pode ser enviado para `Publicado` ou voltar para `Rascunho`.
- Um documento em `Publicado` não pode voltar para nenhum outro estado
- Um documento em `Rascunho` **não pode** ser enviado diretamente para `Publicado`.

E para não perder o costume, vamos ver um exemplo ruim sem a utilização do pattern

editor.py — python

```python
class Documento:
  def __init__(self, titulo: str):
    self._titulo = titulo
    self._estado = "rascunho"  # string solta, sem tipo

  def publicar(self) -> None:
```

```
    if self._estado == "rascunho":
      print("Rascunho enviado para moderação.")
      self._estado = "moderacao"
    elif self._estado == "moderacao":
      print("Aprovado! Documento publicado.")
      self._estado = "publicado"
    elif self._estado == "publicado":
      print("Documento já está publicado.")

  def enviar_para_moderacao(self) -> None:
    if self._estado == "rascunho":
      print("Rascunho enviado para moderação.")
      self._estado = "moderacao"
    elif self._estado == "moderacao":
      print("Documento já está em moderação.")
    elif self._estado == "publicado":
      print("Documento publicado não volta para moderação.")

  def rejeitar(self) -> None:
    if self._estado == "rascunho":
      print("Rascunho não pode ser rejeitado.")
    elif self._estado == "moderacao":
      print("Rejeitado. Voltou para rascunho.")
      self._estado = "rascunho"
    elif self._estado == "publicado":
      print("Documento publicado não pode ser rejeitado.")
```

5.2 A Solução

O State Pattern nos permite modelar esses estados distintos como classes separadas, cada uma representando um estado específico do documento. Dessa forma, comportamentos que variam de acordo com o estado do documento são encapsulados dentro dessas classes de estado, e a classe principal `Documento` delega a responsabilidade de executar comportamentos específicos para o estado atual. Isso mantém o código organizado e facilita a manutenção, além de permitir a adição de novos estados no futuro sem modificar a estrutura existente.

editor.py — python

```
class EstadoDocumento:
  def publicar(self, doc: "Documento") -> None:
    pass

  def enviar_para_moderacao(self, doc: "Documento") -> None:
    pass

  def rejeitar(self, doc: "Documento") -> None:
    pass

```

```
class Rascunho(EstadoDocumento):
  def publicar(self, doc: "Documento") -> None:
    print("Rascunho enviado para moderação.")
    doc.set_estado(Moderacao())

  def enviar_para_moderacao(self, doc: "Documento") -> None:
    print("Rascunho enviado para moderação.")
    doc.set_estado(Moderacao())

  def nome(self) -> str:
    return "Rascunho"


class Moderacao(EstadoDocumento):
  def publicar(self, doc: "Documento") -> None:
    print("Aprovado! Documento publicado.")
    doc.set_estado(Publicado())

  def rejeitar(self, doc: "Documento") -> None:
    print("Rejeitado. Voltou para rascunho.")
    doc.set_estado(Rascunho())

  def nome(self) -> str:
    return "Moderação"


class Publicado(EstadoDocumento):
  def nome(self) -> str:
    return "Publicado"


class Documento:
  def __init__(self, titulo: str):
    self._titulo = titulo
    self._estado: EstadoDocumento = Rascunho()  # depende da abstração

  def set_estado(self, estado: EstadoDocumento) -> None:
    self._estado = estado

  def publicar(self) -> None:
    self._estado.publicar(self)

  def enviar_para_moderacao(self) -> None:
    self._estado.enviar_para_moderacao(self)

  def rejeitar(self) -> None:
```

```
57      self._estado.rejeitar(self)
58
59    @property
60    def estado_atual(self) -> str:
61        return self._estado.nome()
```

Nesse novo código refatorado, cada estado do documento está encapsulado na própria classe representando o próprio comportamento, respeitando o *S* e, para adicionar novos estados, basta criar uma nova classe que herde de `EstadoDocumento`, sem precisar modificar a classe `Documento`. No entanto, é importante notar que o State Pattern possui uma **limitação** no Open Closed Principle. Ao criar um novo estado, por exemplo, um `Arquivado` que eu só posso mudar se meu documento já foi publicado, eu preciso ter que modificar as classes de estado existentes para adicionar a lógica de transição para o novo estado. Querendo ou não, isso é uma violação no OCP, mas é uma violação que é **aceitável** dentro do contexto do State Pattern, pois a adição de novos estados é uma mudança de comportamento **limitada apenas ao contexto dos estados**, para adicionar um novo estado, eu **não preciso modificar a classe `Documento`**, mas eu **preciso** modificar as classes de estado existentes para adicionar a lógica de transição para o novo estado.